

# Formal Analysis of TSN Scheduler for Real-Time Communications

Jin Lv , Yongxin Zhao, *Member, IEEE*, Xi Wu, Yongjian Li, and Qiang Wang 

**Abstract**—Time sensitive networking (TSN) is an emerging technology for in-vehicle networks, which has strict timing constraints and dependability requirements. The performance and efficiency of TSN depend on a reliable scheduling mechanism for real-time communications. Traditionally, the TSN scheduler is analyzed using simulations and testing, which are error-prone and inaccurate. In this article, we employ a timed model checker UPPAAL to formally model and analyze the TSN scheduler with a consideration of its transmission latency and time utilization. We first use the build-in assertion in UPPAAL to verify the deadlock-free property, safety property, and starvation-free property of our model. Then, we calculate the time utilization and the transmission delay of audio video bridging data frame under two scheduling mechanisms according to the simulation process. Then, we compared the time performance of this two scheduling mechanisms and found that each of them has their own advantages and disadvantages. This article provides a formal analysis framework for investigating scheduling strategies in TSN, which facilitates the designers to have a better understanding and development of TSN schedulers in the future.

**Index Terms**—Formal modeling, real-time communications, scheduling mechanism, time sensitive networking (TSN), timing analysis.

## I. INTRODUCTION

WITH the continuous development of the Internet-of-Things, Artificial Intelligence and Automotive Industry, traditional Ethernet is not suitable for real-time and safety-critical applications due to the lack of real-time capabilities.

Manuscript received August 31, 2019; revised March 3, 2020; accepted September 20, 2020. Date of publication October 8, 2020; date of current version August 31, 2021. This article was supported in part by the National Key Research and Development Program of China under Grant 2019YFB2102600, in part by the National Natural Science Foundation under Grant 61672503, in part by the Science and Technology Commission of Shanghai Municipality Project under Grant 18ZR1411600, and in part by the Lab of High Confidence Embedded Software Engineering Technology under Grant HCESET-2019-1. This article was recommended for publication by Associate Editor W. Dong. (Corresponding author: Yongxin Zhao.)

Jin Lv and Yongxin Zhao are with the Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, Shanghai 200241, China (e-mail: lj@stu.ecnu.edu.cn; yxzhao@sei.ecnu.edu.cn).

Xi Wu is with the University of Sydney, Camperdown, NSW 2006, Australia (e-mail: xi.merry.wu@gmail.com).

Yongjian Li is with the State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences, 100864 Beijing, China (e-mail: lyj238@qq.com).

Qiang Wang is with the SUSTech Academy for Advanced Interdisciplinary Studies, Southern University of Science and Technology, Shenzhen 518055, China (e-mail: wenjunwang.nudt@gmail.com).

Color versions of one or more of the figures in this article are available online at <https://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TR.2020.3026689

Therefore, several promising extensions to Ethernet have been proposed, such as TTEthernet [1] and audio video bridging (AVB) [2]. In particular, the *IEEE 802.1 higher layer LAN Protocols Working Group* proposes time sensitive networks (TSNs) by introducing new traffic shapers in IEEE 802.1 to support safety-critical and real-time applications [3].

Since its establishment in 2012, the TSN Working Group has reached many promising achievements [4]. TSN has been applied in many fields [5], [6], including advanced driving assistance system for automobiles [7], [8], which requires networks to have a very high real-time performance. Therefore, most of the research work is dedicated to developing TSN schedulers to meet the urgent needs with high real-time requirements and high reliability [9]. In [10], authors have addressed the computation of fully deterministic schedules for 802.1 Qbv-compliant multihop switched networks. Based on a generalized configuration of parameters, they have derived the constraints for computing offline schedules that can guarantee both low bounded jitters and deterministic end-to-end latency for critical communication flows. In [11], a network calculus-based approach has been proposed to determine the worst-case end-to-end delays of AVB traffic in a TSN network with both nonpreemption and preemption modes. Dorin *et al.* [12] have investigated the effects of time aware shaping on the transmission of AVB traffic in TSN switches. More than that, there are many researchers trying to improve the scheduling mechanism of TSN. In [13], the author proposed sized-based queuing to improve bandwidth utilization in TSN networks that utilize TAS and guard-bands, which can improve efficiency by reducing the amount of wasted bandwidth in a network. However, due to the lack of simulation tools, the author only carried out a quantitative analysis and a method discussion. While focusing on the impact of TSN networks on AVB frames, most researchers apply test-based methods or the worst-case estimation to investigate the worst-case transmission delay of frames, which are error-prone and inaccurate. To the best of our knowledge, there is no research on the formal analysis of scheduling real-time communications in TSNs.

In this article, we present a formal approach in the model checker UPPAAL to analyzing TSN scheduling mechanisms. The TSN scheduler is formalized as a timed automata, whose model consists of three submodels, i.e., traffic model, scheduling table model, and transmission selection model. Besides, we also perform deadlock freedom analysis, safety analysis, and starvation freedom analysis on the TSN scheduler model in UPPAAL. Then, we conduct a formal timing analysis that focuses on time utilization of networks and end-to-end delay of AVB

frames under two different scenarios, which use credit based shaping (CBS) mechanism and first-in-first-out shaper (FIFO) mechanism, respectively.

The main contributions of this article can be summarized as follows.

- 1) *Formal Modeling*: We propose a formal model of the TSN scheduler based on timed automata, which facilitates the designers to thoroughly understand and develop TSN schedulers better. Based on our model, several properties such as deadlock-free property and starvation-free property can be formally verified in the model checker UPPAAL.
- 2) *Timing Analysis*: Our formal analysis considers the time efficiency of scheduling mechanisms and the transmission latency of AVB frames. To the best of our knowledge, this is the first study on the formal analysis of TSN scheduling mechanisms. This method sheds new light on time analysis of TSN schedulers.

The remainder of this article is organized as follows. In Section II, we give a brief introduction to the TSN scheduler and the model checker UPPAAL. In Section III, we present the formal model of the TSN scheduler in UPPAAL. In Section IV, formal analysis of the TSN scheduler and the experimental results are presented. Finally, Section V concludes this article.

## II. PRELIMINARIES

### A. Time-Sensitive Networking

TSN refers to a set of substandards based on specific application requirements under the framework of *IEEE 802.1* standard [4]. The advantage of TSN is to establish a universal time-sensitive mechanism for Ethernet protocols to ensure the time certainty of network data transmission. As the core of TSN standards, *IEEE 802.1 Qbv* introduces the time-triggered traffic shapers to deal with safety-critical and time-sensitive transmissions including control data traffic (CDT), audio/video traffic, (AVB) and best effort traffic (BE) [14]. In general, different traffics have different priorities and the choice of traffic type depends on the applications. These traffics are interpreted in the following:

1) *CDT*: The CDT class refers to the traffics which are time-critical, with strict timing constraints in time-critical applications. In general, CDT traffic is considered to be deterministic as it can be controlled on the sending node. In TSN, CDT flow is assigned a higher priority.

2) *AVB Traffic*: The AVB traffic is originally real-time audio and video streams for studio applications, which motivates the evaluation of Ethernet AVB in industrial automation. In real-time applications, AVB also provides guaranteed time-delay bounds. Different priorities are used for different classes of AVB flows and the priority of AVB traffic is lower than the CDT's.

3) *BE Traffic*: The BE traffic is the basic Ethernet traffic and there are no timing guarantee provided for this traffic type. Thus, the BE traffic is used for sporadic traffic not requiring timing applications.

TSN standard processes each transmitting frame based on a queuing-forwarding mechanism [15]. Therefore, on the output

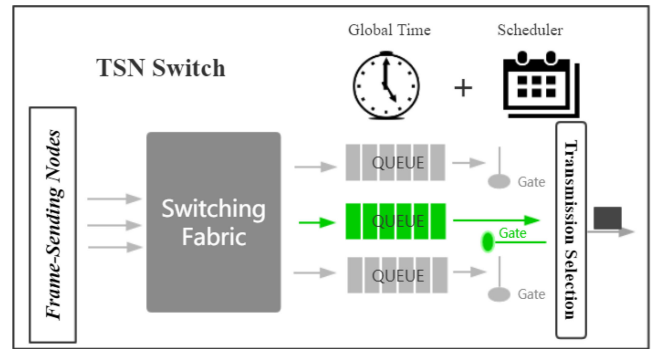


Fig. 1. Architecture of a TSN scheduler.

port of each switch that supports TSN mechanism, it uses queues to store these frames. The scheduler is designed to specify whether the queue is allowed to forward or not by enabling or disabling a transmission gate at each moment in time [16]. A graphical depiction of the architecture of TSN scheduler is shown in Fig. 1.

A TSN period has been divided into different time windows (protected or unprotected) under which different scheduling mechanisms will be applied. We briefly introduce three scheduling mechanisms as follows.

*CBS Mechanism*: The CBS mechanism is employed to prevent starvation of the lower priority queue by introducing a credit as a quota to transmission. According to CBS, an AVB frame is allowed to be transmitted when all the following constraints are satisfied.

- 1) The gate of the queue of AVB frames is enabled.
- 2) There is no other higher priority frame being transmitted.
- 3) The credit of this AVB frame is larger or equal to 0. When multiple credits of queues are greater than or equal to 0, frame with higher priority will be transmitted first.

In CBS mechanism, credits are initially zero. They are decreased at a rate of  $\beta_i^-$  with a send slope while transmitting and frozen while the gate is closed. The credit is increased at a rate of  $\beta_i^+$  with an idle slope when either its credit is negative (and it is not currently transmitting), or when there are messages of class  $i$  queued in the queue but can not transmit because a class of higher priority is transmitting. If the queue is emptied while the credit is positive, the credit is set to zero.

*FIFO Mechanism*: FIFO is an alternative mechanism to schedule transmission frames in the unprotected window. Frames are scheduled according to a FIFO scheduling mechanism. That is, the scheduling is performed in the order of the arrival of the frames among the queues whose gate is open.

*Strict Priority (SP) Mechanism*: This mechanism works under the protected window and only for CDT frames. As the name implies, queues with high priority absolutely take precedence over queues with low priority in this mechanism. As long as there are frames in the queue with high priority, it is scheduled first.

### B. UPPAAL Model Checker

UPPAAL [17], [18] is a well-established model checker for modeling, simulation, and verification of real-time systems. It

is widely used in both academia and industry [19], [20], including the areas of avionics, ambient intelligence, and emergency contingencies.

In UPPAAL, a system can be modeled as a collection of nondeterministic processes with real-valued clocks, in which the communication between each process happens via either channels or shared variables. In synchronization channels, one process is synchronized with exactly with one other process, whereas for shared variables, one process is synchronized with all other processes using the shared variables. Each process can be modeled as a timed automata  $M_{st} = (Q, C, \Sigma, E, I, q_0)$ , where

- 1)  $Q$  is the set of states;
- 2)  $C$  represents the set of clocks;
- 3)  $\Sigma$  is the set of actions over edges which can be recognized by the automata and may change the value of some variables or reset clocks;
- 4)  $E \subseteq Q \times \beta(C) \times \Sigma \times 2^C \times Q$  is the set of edges, where  $\beta(C)$  is the set of conjunctions over simple conditions of clock constraints and  $2^C$  is the set of clocks to be reset;
- 5)  $I : Q \rightarrow \beta(C)$  is a mapping of invariant on states;
- 6)  $q_0$  is the initial state of the automata.

UPPAAL also provides tools for both validation (via graphical simulation) and verification (via model checking) of real-time systems. The simulator may detect possible errors in the system model via providing a dynamic execution of the system, whereas the verifier checks the properties including reachability, safety, and liveness, which are described by the computational tree logic (CTL), of the model by on-the-fly searching the system state space. The CTL provides the formal specification of the described system properties using logical, temporal, and path operators. The logical operators are conjunction (&), disjunction ( $\parallel$ ), negation (!), implication ( $\rightarrow$ ), and equality ( $\leftrightarrow$ ). The temporal operators are always ( $\square$ ), eventually ( $\diamond$ ), next ( $\mathbf{X}$ ) and until ( $\mathbf{U}$ ), whereas for all ( $\mathbf{A}$ ) and there exists ( $\mathbf{E}$ ) are the path operators.

### III. FORMALIZATION IN UPPAAL

In this section, we formally model the TSN scheduler in UPPAAL, consisting of the overview model of the TSN scheduler, traffic model, scheduling table model, and transmission selection model.

#### A. TSN Scheduler Model

This section describes the overview model of the TSN scheduler, whose framework can be found in Fig. 2. The model mainly consists of three parts: traffic model, scheduling table model, and transmission selection model. The traffic model is composed of three types of traffic, i.e., CDT, AVB (i.e., AVB\_A and AVB\_B), and BE. The scheduling table model defines different types of time windows (i.e.,  $S_0$ ,  $S_1$ , and  $S_2$ ) for each period in a time division multiple access (TDMA) fashion and this period repeats for the entire life-time of the system, and we will describe these time windows below. Thus, different traffics would be isolated and scheduled in their own time windows. The transmission selection model describes the transmission

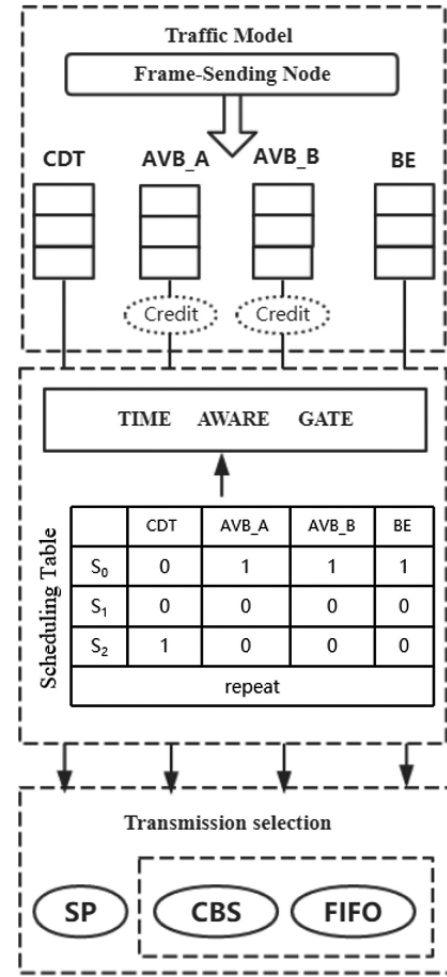


Fig. 2. Framework of TSN scheduler model.

scheduling mechanisms, which include CBS mechanism, FIFO mechanism, and SP mechanism.

The TSN scheduler is modeled as a UPPAAL process, whose behavior is described by parallelizing its three subprocesses, i.e., traffic model, scheduling table model, and transmission selection model, as shown as follows:

```

system traffic_Model,
      schedule_Table_Model,
      trans_Selection_Model;

```

In the traffic model, we define the frame-sending nodes to generate the frames, which will be stored in queues. These frame-sending nodes, together with the scheduling table model and the transmission selection model, will be defined as time automata in UPPAAL and they will share the global clock  $t_{global}$ . Communications between time automata are synchronously over channels and via shared global variables. More details of each model can be found in the following sections.

#### B. Traffic Model

Real-time communications in TSN are divided and wrapped in several Ethernet frames, which are transmitted in a sequence from the source to the destination. The traffic model defines the frame-sending nodes to generate frames. And the definition of frame and queue will also be given as follows.

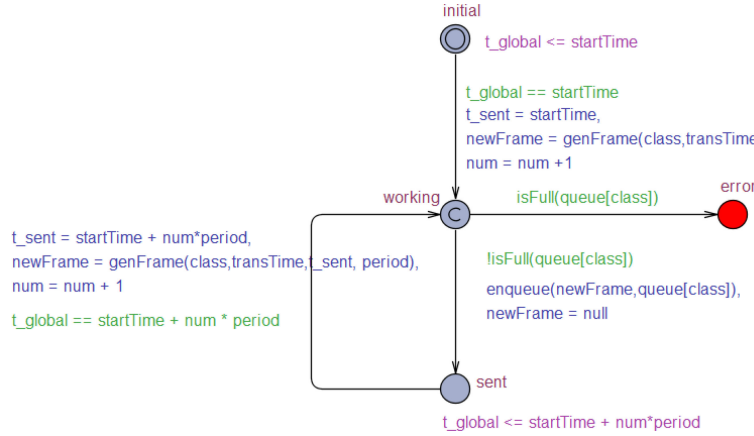


Fig. 3. Model of frame-sending node.

**Definition 1 (Frame):** A data frame  $F$  is defined as a quadruple  $(class, transTime, timestamp, period)$ , where

- 1)  $class$  defines the type of the frame, i.e., CDT, AVB\_A, AVB\_B, and BE;
- 2)  $transTime$  stands for the transmission duration of the frame, i.e., the length of the frame;
- 3)  $timestamp$  represents the arrival time instant of the frame;
- 4)  $period$  represents the cycle for a frame-sending node to generate one frame;

The data frame  $F$  could be represented as a data structure *Frame* in UPPAAL, the detail of which is given as follows:

```

typedef struct{
    int class;
    int transTime;
    int timestamp;
    int period;
} Frame;
  
```

We formalize the frame-sending node as a timed automata shown in Fig. 3. In this model, there are four states including *initial*, *working*, *sent*, and *error*. The variable  $t\_global$  represents the global clock synchronized by all processes. Besides, there are also some other variables and functions defined as follows:

- 1)  $startTime$  indicates the starting working time of the frame-sending node;
- 2)  $t\_sent$  represents the time when a frame is generated (and sent), whose value will be regarded as the timestamp of the generated frame;
- 3)  $num$  is to record the number of frames that have been generated;
- 4)  $genFrame(\dots)$  is a function used to generate frames;
- 5)  $isFull(\dots)$  is a function that determines whether a queue is full;
- 6)  $enqueue(\dots)$  is a function, which is responsible for enqueueing frames.

The frame-sending node model is responsible for generating frames, which will be transmitted later. In the *initial* state, if the global clock  $t\_global$  equals to the value of  $startTime$ , the frame-sending node starts to generate the first frame. At the same time, the number of total frames  $num$  increases, and the

state changes from initial state to *working* state. Later, the state continues transferring among *working* state, *sent* state, and *error* state according to the guard conditions. Particularly, when the relevant queue, which stores frames is full, the frame-sending node model may move to the *error* state from the current *working* state, and it stops at this *error* state.

The generated frames are organized in a queue. Different frames are deployed in different queues according to their own class. The priority of a queue is corresponding to the class of frames deployed in the queue. In TSN standard, there are up to eight different priorities of queues. Thus, a queue is defined as follows:

**Definition 2 (Queue):** A queue  $Q$  is a quadruple in the form of  $(prior, front, rear, seq[n])$ , where,

- 1)  $prior$  defines the priority of the queue according to the class of its frames and  $prior \in [0..7]$ ;
- 2)  $front$  describes the index of the first element of the queue;
- 3)  $rear$  represents the index of the last element in the queue;
- 4)  $seq[n]$  stands for a sequence of frames, and its length is  $n$ .

A queue is represented as an instance of the data structure *FrameQ*, which in UPPAAL is modeled as

```

typedef struct {
    int prior;
    int front;
    int rear;
    Frame seq[n];
} FrameQ;
  
```

### C. Scheduling Table Model

In TSN standards, each time period is divided into unprotected time window and protected time window in a TDMA fashion, which is illustrated in Fig. 4. Different kinds of traffics are isolated and scheduled in their own time windows. The protected time window is used to transmit time-critical traffic (i.e., CDT frames) and the unprotected time window is used for other traffics (i.e., AVB and BE frames). In order to ensure that nontime-critical traffics cannot affect the transmission of time-critical traffic, a guard band is introduced to restrict new transmission of nontime-critical traffics. Thus, in the worst case, the guard band should be as long as a maximum-sized frame.

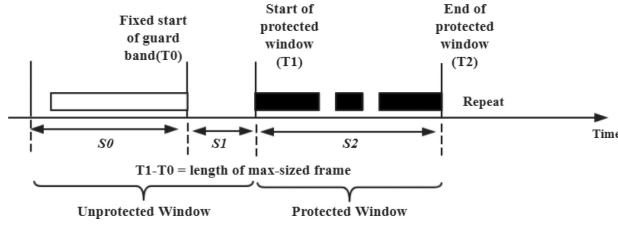


Fig. 4. Example of TSN time windows configuration.

Therefore, a time period is split into three slots, i.e., nontime-critical traffic slot  $S_0$ , guard band slot  $S_1$ , and time-critical traffic slot  $S_2$ , as shown in Fig. 4.

Besides, several time aware gates deployed for traffic queues are employed to control the transmission of frames according to a predefined scheduling table. For each slot of time period, the scheduling table specifies whether different traffics can be transmitted, respectively. An example of scheduling table can be found in Fig. 2, where 1 indicates that the transmission gate is enabled while 0 represents that it is disabled.

Next, we formalize the scheduling table in UPPAAL as a timed automata shown in Fig. 5. There are six states in this model, namely *initial* state, *unpro\_win* state standing for unprotected window, *unpro\_win\_gb* state indicating the guard band, *pro\_win* state representing the protected window, and two transition states  $s\_1$  and  $s\_2$ .

Besides, there are also some other variables defined in this model

- 1)  $slot[n]$  is an array recording the length of each time window;
- 2)  $slot\_id$  indicates the index of time slots;
- 3)  $t\_slot$  represents the variable who records the time passing in the time window;
- 4)  $total\_time$  is to record the total time from the beginning of this cycle to the end of the current time slot, which could help determine whether the remaining time in the current time window can support the frame to be transmitted;
- 5)  $flag\_gb$  and  $flag$  are boolean variables. The former indicates whether the guard band is entered and the latter indicates whether there is a frame being transmitted;
- 6)  $method[S]$  is a channel array which is used for synchronization and  $S$  is the name of a scheduling mechanism and  $S \in \{CBS, FIFO, SP\}$ .

The scheduling table model can help us determine which time window the current time is located in. It will call the corresponding transmission selection algorithm by sending synchronization signals via the channel  $method[S]!$  to other related automata.

#### D. Transmission Selection Model

Three transmission selection mechanisms are adopted in this article. Similar to the scheduling table, these three transmission selection mechanisms are all modeled as time automata in UPPAAL, whose details are explained as follows.

1) *Model of CBS Mechanism*: The CBS mechanism works under the unprotected window and only works on the AVB\_A and AVB\_B frames. In Section II, we have already introduced

the way the CBS mechanism works. Here, we mainly give the UPPAAL model of CBS mechanism in the following.

The timed automata of CBS mechanism is shown in Fig. 6. As we can see, there are four states, namely initial state *initial*, ready state *ready*, transmission start state *startTrans*, and transmission end state *transmitted*. Here, we define a local clock variable *time* to indicate the elapse of time during the transmission of frames. When the value of *time* equals to the transmission time of the selected frame, the transmission is completed.

The other variables of this model are defined as follows:

- 1)  $f\_class$  is simply used to record the type of frame being transmitted, whose value belongs to  $\{AVB\_A, AVB\_B, BE\}$ .
- 2)  $credit[j]$ ,  $j \in \{0, 1\}$  is an array, whose element represents the credit of the queue of AVB frames, where  $j = 1$  indicates the credit of AVB\_A while  $j = 0$  indicates the credit of AVB\_B.
- 3)  $tran[k]$ ,  $k \in \{AVB\_A, AVB\_B, BE\}$  is an array of channels, via which timed automata can communicate with each other. The channel can also illustrate what type of frames is being transmitted.
- 4)  $tr[j]$ ,  $j \in \{CDT, AVB\_A, AVB\_B, BE\}$  is an array used to record the number of frames that each queue has forwarded. In the CBS scheduling model and FIFO scheduling model, the value of  $j$  can only take the last three while in SP scheduling model takes the first one.

The behavior of the automata is processed among four states. In the *initial* state, if the synchronization signal of the scheduling table is received, and there is a nonempty queue whose transmission gate is open, the process invokes the CBS transmission selection mechanism to select a frame to be transmitted, and the state goes to the ready state *ready*.

When the state *ready* is reached, it will check whether the current time window allows the selected frame to continue to be transmitted. If the time is insufficient, it returns to the *initial* state and waits for the next time window. If the time is sufficient, then the system will assign the value of *flag* to true and the value of  $f\_class$  to the type of the selectedFrame. The state then moves to *startTrans*.

In the state *startTrans*, the system satisfies the invariant  $time \leq selectedFrame.trans\_delay$ . When the value of *time* equals to the transmission time of the selected frame, the frame transmission is complete and the state goes to *transmitted*. At the same time, the synchronization signal  $tran[k]!$  ( $k \in \{AVB\_A, AVB\_B, BE\}$ ) is sent to the automata of credit controller to make an action changing the credit value of the corresponding queue.

At the end of the transmission, if there exists nonempty queue whose transmission gate is open, then the system will continue to select the next frame that may be transmitted, and the state goes to *ready*. Otherwise the state goes to the initial state *initial*.

During the transmission of frames, the change of the credit variable *credit* of the corresponding queue is controlled by an automata, which is shown in Fig. 7

- 1)  $inc\_Credit(\dots)$  is a function responsible for the *idleslope* to increase the credit of AVB queues with an increase rate  $\beta^+$ ;

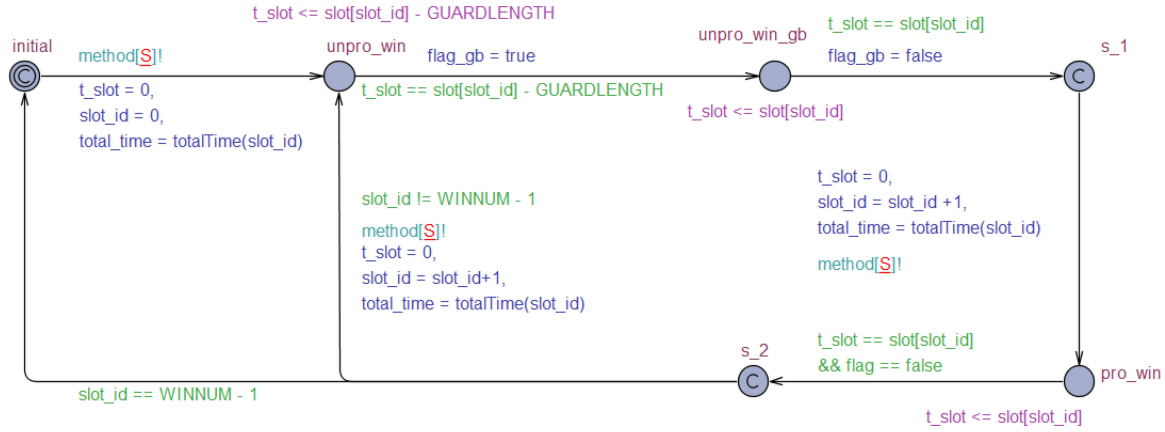
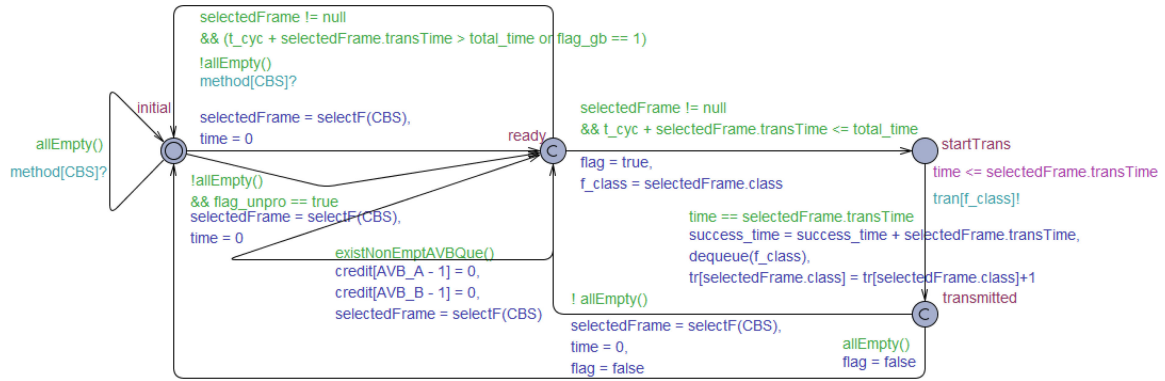
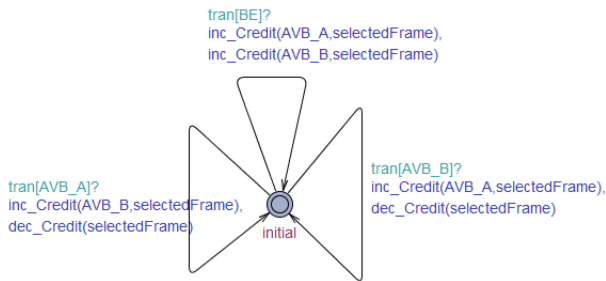
Fig. 5. Scheduling table model (abbr. *schTab*).Fig. 6. Model of CBS mechanism (abbr. *cbs*).

Fig. 7. Model of credit controller.

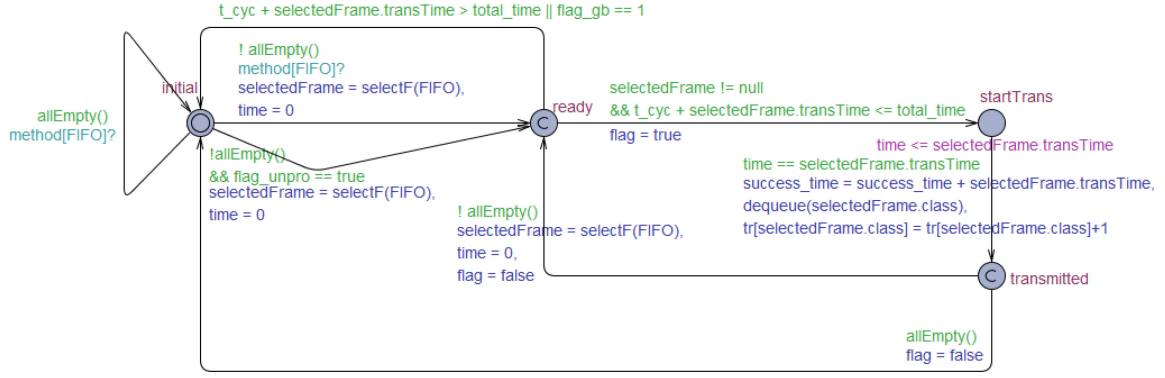
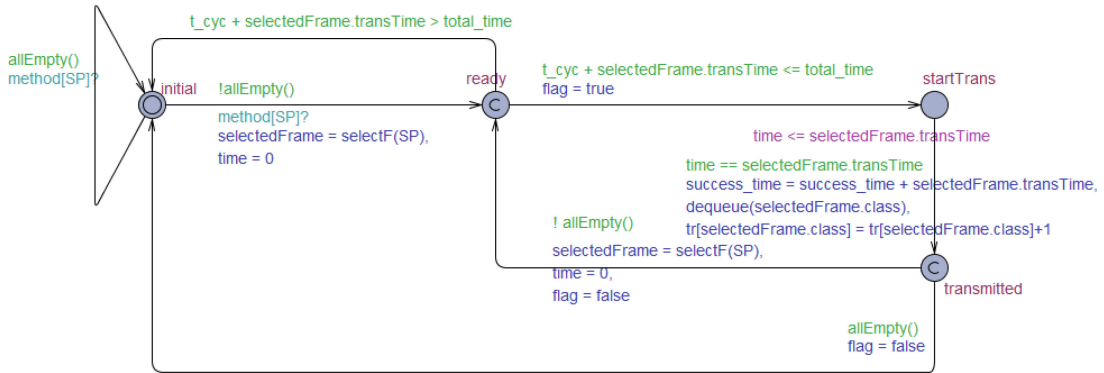
2) *dec\_Credit(...)* is a function responsible for the *sendslope* to decrease the credit of AVB queues with a decrease rate  $\beta^-$ .

2) *Model of FIFO Mechanism*: The FIFO policy is the same as the CBS mechanism, and they all work under unprotected windows. Compared with CBS, the FIFO policy does not consider the priority of the scheduling process, and is fairer for low priority frames, but has no advantage for higher priority frames. We discuss this mechanism in UPPAAL as follows.

This timed automata is similar to that of CBS mechanism and is shown in Fig. 8. Like what we have seen, the state set of FIFO mechanism model is the same as that of CBS mechanism model. We also define a local clock variable *time* to denote the elapse of time during the transmission of frames. The most difference is that when the synchronization signal of the scheduling table model is received, and there is a nonempty queue with transmission gate open, the process will call the FIFO transmission selection mechanism to select a frame to be transmitted and there is no change in the value of *credit* because the FIFO mechanism does not involve changes in credit of AVB frames. The state transition process of this model is similar to that of CBS model.

3) *Model of SP Mechanism*: The SP mechanism works under the protected window and only works on CDT frames. The time automata of SP mechanism is analogous to that of the other two mechanisms, which can be found in Fig. 9.

The state set of this mechanism is the same as the above two mechanisms. The local variable *time* represents the time passing during the transmission of frames. There is no change in the value of the *credit*. The only difference is that when the synchronization signal of the Scheduling table model is received, and there is a nonempty queue with transmission gate open, the process will call the SP transmission selection mechanism to

Fig. 8. Model of FIFO mechanism (abbr. *fifo*).Fig. 9. Model of SP mechanism (abbr. *sp*).TABLE I  
PARAMETERS OF TRAFFIC CLASSES

| Class     | CDT         | AVB_A      | AVB_B      | BE         |
|-----------|-------------|------------|------------|------------|
| Size      | 175B        | 325B       | 225B       | 275B       |
| transTime | 14 $\mu s$  | 26 $\mu s$ | 18 $\mu s$ | 22 $\mu s$ |
| Period    | 200 $\mu s$ | 20 $\mu s$ | 30 $\mu s$ | 30 $\mu s$ |
| Priority  | 3           | 2          | 1          | 0          |
| $\beta^+$ | -           | 6          | 8          | -          |
| $\beta^-$ | -           | 8          | 6          | -          |

select a frame to be transmitted. Then, the transition process is similar to the other two mechanism models.

#### IV. EXPERIMENT AND FORMAL ANALYSIS

In this section, we describe the main results of our experiments and analyze the time performance of different TSN scheduling mechanisms.

##### A. Experimental Configuration

In our experimental studies, we consider four classes (i.e., four types) of data frames, including CDT, AVB\_A, AVB\_B, and BE. Table I shows the parameters of these four traffic classes, whose explanations can be found as follows:

- 1) *Size* indicates the size of the frame in bytes.

- 2) *transTime* denotes the transmission time (or the length) of the frame, which is calculated by the size of the frame and the network bandwidth. In this article, we assume the bandwidth is 100 Mbps.
- 3)  $\beta^+$  and  $\beta^-$  represent the idle-slope and send-slope of AVB frames, respectively.

According to IEEE 802.1 TSN [3], [21], the length of a TSN period is 500  $\mu s$ , in which the length of each protected window equals to the length of a CDT frame, that is,  $L_{Pro} = 14 \mu s$ . In our model, we define two protected time windows and two unprotected time windows with guard bands in one period. For simplicity, we define a fixed-length guard band, whose length equals to the max length of frames, thus  $L_{gb} = 26 \mu s$ . The remaining length of a period can be used for the transmission of AVB frames and BE frames.

The scheduling process will use different scheduling mechanisms to schedule these frames and we will analyze the time utilization of each scheduling mechanism the scheduling process uses.

##### B. Formal Verification and Timing Analysis

We conduct two groups of experiments with two different scheduling mechanisms. The difference of the two experiments is that under unprotected time window one is using CBS

scheduling mechanism and the other is using FIFO scheduling mechanism, while in protected time window both using SP scheduling mechanism.

1) *Formal Verification*: Before we do the timing analysis, we are going to verify whether the system model satisfies the properties below via the build-in assertions in UPPAAL.

*Property 1: Deadlock-free Property*

$A[] \text{ not deadlock}$

This property guarantees that the model would not progress into a deadlock situation, which is satisfied in our model.

*Property 2: Mutually Exclusive*

$A[] \text{ not}(sp.startTrans \& \& cbs.startTrans)$

$A[] \text{ not}(sp.startTrans \& \& fifo.startTrans)$

The above two formulas guarantee that different scheduling mechanism will not transmit frames at the same time, which ensures the specificity of dedicated bandwidth resources.

*Property 3: Frame Transmission Without Crossing Time Windows*

$A[] sp.startTrans \text{ imply } sp.time \leq selectedFrame.transTime$

$A[] cbs.startTrans \text{ imply } cbs.time \leq selectedFrame.transTime$

$A[] fifo.startTrans \text{ imply } fifo.time \leq selectedFrame.transTime$

The above three formulas guarantee that if the scheduling mechanism can transmit frames, it must be able to complete the transmission within the specific time window. They ensure that the transmission of noncritical frames does not occupy the protected time window which is used to transmit time-critical frames.

*Property 4: Frame-sending Node Will Work All The Time*

$A[] \text{ not}(senderBE.error \parallel senderAVB\_B.error \parallel senderAVB\_A.error \parallel senderCDT.error)$

This formula means that for all path sequences, the frame-sending node model will not enter into the *error* state. As we can see from Fig. 10, the above formula is not satisfied. It illustrates that the queue in which the frames are stored may be full. This is because the length of queues is fixed, which may be a limitation of our model.

*Property 5: Starvation-Free Properties*

$A <> fifo.ready \text{ imply } fifo.startTrans$

$A <> sp.ready \text{ imply } sp.startTrans$

$A <> cbs.ready \text{ imply } cbs.startTrans$

These three formulas guarantee that the model of SP (or CBS/FIFO) mechanism will eventually start transmitting frames after they go to *ready* state. The verification results can be found in Fig. 10.

2) *Timing Analysis*: We analyze the time performance on these two groups of experiments, especially focusing on the delay analysis of AVB frames, whose results can be found in Tables II–IV. In Table II,  $T_{t\_global}$  represents the value of global clock  $t\_global$  after all frames has been transmitted, and  $T_{sum}$  stands for the total transmission time of all frames, which can be calculated via the following formula:

$$T_{sum} = \sum_{j=1}^N transTime_j$$

| Status                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Established direct connection to local server.<br>(Academic) UPPAAL version 4.1.22 (rev. 5FDF013C22EB4717), March 2019 -- server.<br>A[] not deadlock<br>Verification/kernel/elapsed time used: 0s / 0s / 0.012s.<br>Resident/virtual memory usage peaks: 12,224KB / 34,672KB.<br>Property is satisfied.                                                                                                                                                                                                                                                                                                                                   |
| A[] not (sp.startTrans && fifo.startTrans)<br>Verification/kernel/elapsed time used: 0s / 0s / 0.008s.<br>Resident/virtual memory usage peaks: 12,176KB / 34,416KB.<br>Property is satisfied.<br>A[] sp.startTrans imply sp.time <= selectedFrame.trans_delay<br>Verification/kernel/elapsed time used: 0s / 0s / 0.02s.<br>Resident/virtual memory usage peaks: 12,196KB / 34,460KB.<br>Property is satisfied.<br>A[] fifo.startTrans imply fifo.time <= selectedFrame.trans_delay<br>Verification/kernel/elapsed time used: 0.015s / 0s / 0.032s.<br>Resident/virtual memory usage peaks: 12,208KB / 34,484KB.<br>Property is satisfied. |
| A[] cbs.startTrans imply cbs.time <= selectedFrame.trans_delay<br>Verification/kernel/elapsed time used: 0.015s / 0s / 0.031s.<br>Resident/virtual memory usage peaks: 13,680KB / 37,568KB.<br>Property is satisfied.<br>A[] not (sp.startTrans && cbs.startTrans)<br>Verification/kernel/elapsed time used: 0.016s / 0s / 0.011s.<br>Resident/virtual memory usage peaks: 13,700KB / 37,600KB.<br>Property is satisfied.                                                                                                                                                                                                                  |
| A[] not (senderBE.error    senderAVB_B.error    senderAVB_A.error    senderCDT.error)<br>Verification/kernel/elapsed time used: 0.672s / 0.016s / 0.678s.<br>Resident/virtual memory usage peaks: 113,796KB / 239,476KB.<br>Property is not satisfied.                                                                                                                                                                                                                                                                                                                                                                                     |
| A<> fifo.ready imply fifo.startTrans<br>Verification/kernel/elapsed time used: 0s / 0s / 0.012s.<br>Resident/virtual memory usage peaks: 12,208KB / 34,484KB.<br>Property is satisfied.<br>A<> sp.ready imply sp.startTrans<br>Verification/kernel/elapsed time used: 0s / 0s / 0.017s.<br>Resident/virtual memory usage peaks: 11,268KB / 33,556KB.<br>Property is satisfied.<br>A<> cbs.ready imply cbs.startTrans<br>Verification/kernel/elapsed time used: 0s / 0s / 0.013s.<br>Resident/virtual memory usage peaks: 12,688KB / 36,596KB.<br>Property is satisfied.                                                                    |

Fig. 10. Verification results.

TABLE II  
TIME PERFORMANCE ANALYSIS

| Mechanisms | $T_{t\_global}$ | $T_{sum}$    | $U$    |
|------------|-----------------|--------------|--------|
| CBS        | 1000 $\mu$ s    | 928 $\mu$ s  | 92.8%  |
|            | 2000 $\mu$ s    | 1856 $\mu$ s | 92.8%  |
|            | 3000 $\mu$ s    | 2784 $\mu$ s | 92.8%  |
|            | 4000 $\mu$ s    | 3712 $\mu$ s | 93.0%  |
|            | 5000 $\mu$ s    | 4656 $\mu$ s | 93.12% |
| FIFO       | 1000 $\mu$ s    | 956 $\mu$ s  | 95.6%  |
|            | 2000 $\mu$ s    | 1930 $\mu$ s | 96.5%  |
|            | 3000 $\mu$ s    | 2886 $\mu$ s | 96.2%  |
|            | 4000 $\mu$ s    | 3824 $\mu$ s | 95.6%  |
|            | 5000 $\mu$ s    | 4780 $\mu$ s | 95.6%  |

where  $N$  is the number of frames.  $U$  is the time utilization, which can be calculated via formula

$$U = \frac{T_{sum}}{T_{t\_global}} \times 100\%$$

In Table II, we calculate the time utilization when  $t\_global = T$  time units, where  $T = 1000, 2000, \dots, 5000$  time units. It shows that the time utilization of using CBS is slightly lower than that of using FIFO. When the payloads increases, the time efficiency of the two mechanisms does not decrease too much, and tends to be stable.

TABLE III  
DELAY ANALYSIS OF AVB FRAMES

| Class/Mechanism   | $maxDelay$   |
|-------------------|--------------|
| <i>AVB_A/CBS</i>  | 2788 $\mu s$ |
| <i>AVB_A/FIFO</i> | 3168 $\mu s$ |
| <i>AVB_B/CBS</i>  | 2480 $\mu s$ |
| <i>AVB_B/FIFO</i> | 3130 $\mu s$ |

TABLE IV  
FRAMES FORWARDED WITHIN TEN CYCLES

| Mechanism/Class | <i>CDT</i> | <i>AVB_A</i> | <i>AVB_B</i> | <i>BE</i> |
|-----------------|------------|--------------|--------------|-----------|
| <i>CBS</i>      | 20         | 102          | 96           | 2         |
| <i>FIFO</i>     | 20         | 90           | 60           | 60        |

The delay of a frame represents the time a frame waits to be transmitted (i.e., starting transmission time of a frame subtracts the time it arrives at the queue). In Table III,  $maxDelay$  represents the maximal transmission delay of AVB frames. A conclusion can be drawn from Table III is that the delay of AVB frames in using CBS mechanism is much lower than that in using FIFO mechanism.

Besides, we also count the number of the four types of frames forwarded within 10 cycles in Table IV. In Table IV, we can see that the number of *AVB* frame forwarded by CBS scheduling mechanism is obviously more than the number of *BE* frame during this period, whereas the number of *AVB\_B* frame and *BE* frame forwarded by FIFO is almost the same. This is because the CBS mechanism essentially transmits AVB frames prior to BE frames, whereas FIFO mechanism transmits frames according to the arrival time of frames without preference.

## V. CONCLUSION

In this article, we employed the timed model checker UPPAAL to model and analyze different scheduling mechanisms in TSN. The TSN schedule was modeled as the system process, which contains several scheduling processes running in parallel. Based on the model, several properties such as deadlock-free property, safety property, and starvation-free property have been verified to make sure the system runs normally. Besides, we also conduct a timing analysis on different scheduling mechanisms used in the unprotected time windows. In particular, we compare the time utilization and the delay of AVB frames in two cases. The result showed that the time utilization of using the CBS scheduling mechanism in the unprotected time windows was slightly lower than using the FIFO scheduling mechanism, but the CBS mechanism has a better performance than FIFO mechanism in terms of the transmission delay of AVB frames. Therefore, it can be seen that different scheduling mechanisms have their own advantages and disadvantages. Through our framework, designers can easily achieve a better understanding of the TSN schedulers they designed, which may guide their further development.

Due to the limitation of experimental conditions, we only analysis the time performance of our model in a certain period of time. In the future, we will work towards the optimization of our model to break this limitation. We will also try to investigate

some other scheduling mechanisms such as preemption in the scheduling process of TSN.

## REFERENCES

- [1] W. Steiner, G. Bauer, B. Hall, M. Paulitsch, and S. Varadarajan, "Ttethernet dataflow concept," in *Proc. 8th IEEE Int. Symp. Netw. Comput. Appl.*, 2009, pp. 319–322.
- [2] L. L. Bello, "Novel trends in automotive networks: A perspective on ethernet and the ieee audio video bridging," in *Proc. Emerg. Technol. Factory Autom.*, 2014, pp. 1–8.
- [3] "Ieee standard for local and metropolitan area network-bridges and bridged networks," *IEEE Std 802.1Q-2018 (Revision IEEE Std 802.1Q-2014)*, pp. 1–1993, Jul. 2018.
- [4] J. Farkas, L. L. Bello, and C. Gunther, "Time-sensitive networking standards," *IEEE Commun. Standards Mag.*, vol. 2, no. 2, pp. 20–21, Jun. 2018.
- [5] D. Bruckner *et al.*, "An introduction to OPC UA TSN for industrial communication systems," *Proc. IEEE*, vol. 107, no. 6, pp. 1121–1131, Jun. 2019.
- [6] D. Bruckner *et al.*, "An introduction to OPC UA TSN for industrial communication systems," *Proc. IEEE*, vol. 107, no. 6, pp. 1121–1131, Jun. 2019. [Online]. Available: <https://doi.org/10.1109/JPROC.2018.2888703>
- [7] J. Lee and S. Park, "Time-sensitive network (TSN) experiment in sensor-based integrated environment for autonomous driving," *Sensors*, vol. 19, no. 5, 2019, Art. no. 1111. [Online]. Available: <https://doi.org/10.3390/s19051111>
- [8] E. Gardiner, "The AVNU alliance theory of operation for TSN-enabled industrial systems," *IEEE Commun. Standards Mag.*, vol. 2, no. 1, pp. 5–5, Mar. 2018.
- [9] F. Smirnov, M. Glaŷ, F. Reimann, and J. Teich, "Formal timing analysis of non-scheduled traffic in automotive scheduled TSN networks," in *Des., Autom. Test Eur. Conf. Exhibit.*, Mar. 2017, pp. 1643–1646.
- [10] S. S. Craciunas, R. S. Oliver, M. Chmelfk, and W. Steiner, "Scheduling real-time communication in IEEE 802.1 QBV time sensitive networks," in *Proc. 24th Int. Conf. Real-Time Netw. Syst.*, 2016, pp. 183–192.
- [11] L. Zhao, P. Pop, Z. Zheng, and Q. Li, "Timing analysis of AVB traffic in TSN networks using network calculus," in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp.*, Apr. 2018, pp. 25–36.
- [12] D. Maxim and Y. Q. Song, "Delay analysis of AVB traffic in time-sensitive networks (TSN)," in *Proc. 25th Int. Conf.*, 2017, pp. 18–27.
- [13] F. Heilmann and G. Fohler, "Size-based queuing: An approach to improve bandwidth utilization in TSN networks," *SIGBED Rev.*, vol. 16, no. 1, pp. 9–14, 2019. [Online]. Available: <https://doi.org/10.1145/3314206.3314207>
- [14] D. Gizopoulos, D. Alexandrescu, M. Maniatakis, and P. Papavramidou, Eds., in *Proc. 24th IEEE Int. Symp. On-Line Testing Robust Syst. Des., IOLTS 2018, Platja D'Aro, Spain, Jul. 2–4, 2018*, 2018. [Online]. Available: <http://ieeexplore.ieee.org/xpl/mostRecentIssue.jsp?punumber=8450543>
- [15] *IEEE Standard for Local and metropolitan area networks – Virtual Bridged Local Area Networks Amendment 12: Forwarding and Queuing Enhancements for Time-Sensitive Streams*, 2010, pp. 1–72. [Online]. Available: <http://ieeexplore.ieee.org/xpl/mostRecentIssue.jsp?punumber=8233358>
- [16] A. Nasrallah *et al.*, "Performance comparison of IEEE 802.1 TSN time aware shaper (TAS) and asynchronous traffic shaper (ATS)," *Access*, vol. 7, pp. 44 165–44 181, 2019. [Online]. Available: <https://doi.org/10.1109/ACCESS.2019.2908613>
- [17] G. Behrmann, A. David, and K. G. Larsen, "A Tutorial on Uppaal," in *Proc. Int. School Formal Methods Des. Real-Time Syst.*, ser. Lecture Notes in Computer Science, vol. 3185, 2004, pp. 200–236.
- [18] J. Bengtsson, K. G. Larsen, F. Larsson, P. Pettersson, and W. Yi, "UPPAAL – a Tool Suite for Automatic Verification of Real-Time Systems," in *Proc. DIMACS/SYCON Workshop Verification Control Hybrid Syst.III*, ser. Lecture Notes in Computer Science, vol. 1066, 1995, pp. 232–243.
- [19] K. G. Larsen, F. Lorber, and B. Nielsen, "20 Years of UPPAAL Enabled Industrial Model-Based Validation and Beyond," in *Proc. 8th Int. Symp. Leveraging Appl. Formal Methods, Verification Validation, ISOla 2018*, ser. Lecture Notes in Computer Science, vol. 11247, 2018, pp. 212–229.
- [20] S. Saini and A. Fehnker, "Evaluating the stream control transmission protocol using Uppaal," in *Proc. 2nd Workshop Models Formal Anal. Real Syst., MARS@ETAPS 2017*, ser. EPTCS, vol. 244, 2017, pp. 1–13.
- [21] N. Concer, S. Thangamuthu, P. Cuijpers, and J. Lukkien, "Analysis of ethernet-switch traffic shapers for in-vehicle networking applications," in *Proc. Des., Autom. Test Eur. Conf. Exhibit.*, 2015, pp. 55–60.